

Manual Técnico

Sistema de Gestión Hotelera — Paradise Resort

Versión 1.0

Universidad Latina de Costa Rica Ingeniería en Sistemas Computacionales Análisis y Diseño de Sistemas II

Profesor: Lic. Christopher Seas

Autores

Marcos Vargas Borge · Sebastián José Mora Ortiz · Josué Andrés Rojas Vásquez Sebastián José Rojas Quirós
· Yesica Madriz Chaves

Fecha: julio de 2026

Tecnologías: ASP.NET Core 8 · Entity Framework Core 8 · SQL Server · React 19 · TypeScript · Vite · Material
UI

Índice

1. Introducción
 2. Arquitectura
 3. Tecnologías y justificación
 4. Base de datos
 5. Backend
 6. Frontend
 7. Patrones de diseño
 8. Seguridad
 9. Pruebas
 10. Instalación
 11. Despliegue
 12. Mantenimiento
-

1. Introducción

1.1 Descripción del sistema

Paradise Resort administraba sus reservaciones mediante hojas electrónicas y registros manuales. Ese modelo provocaba sobre-reservas, ausencia de visibilidad sobre la disponibilidad real de habitaciones y, con frecuencia, consumos de servicios que no llegaban a la factura del huésped. Las solicitudes de reserva, además, llegaban dispersas por teléfono, correo electrónico y redes sociales sin un punto único de registro.

El sistema centraliza esa operación en una plataforma web que cubre el ciclo completo del huésped: registro del cliente, consulta de disponibilidad, reserva, check-in, consumo de servicios adicionales, check-out y facturación consolidada, más los reportes gerenciales de ocupación, ingresos y temporadas.

1.2 Objetivos

Objetivo	Cómo lo resuelve el sistema
Eliminar las sobre-reservas	Verificación de disponibilidad dentro de una transacción serializable, con bloqueo optimista por fila
Dar visibilidad de la disponibilidad	Panel con el estado de cada habitación en tiempo real y consulta por rango de fechas
Evitar fugas por consumos no cobrados	Los consumos se imputan a la estadía y la factura los incorpora automáticamente
Agilizar el check-out	Cierre de estadía y emisión del comprobante en una sola operación
Garantizar la confidencialidad	Autenticación JWT y autorización por rol
Permitir crecimiento futuro	Arquitectura en capas con una fachada de servicios preparada para app móvil y plataformas OTA

1.3 Alcance

Cubre los nueve requerimientos funcionales especificados en la Etapa 1 (RF01–RF09) y los siete no funcionales (RNF01–RNF07). Quedan fuera del alcance actual, aunque previstas por la arquitectura, la aplicación móvil para huéspedes y la integración con plataformas externas de reservación.

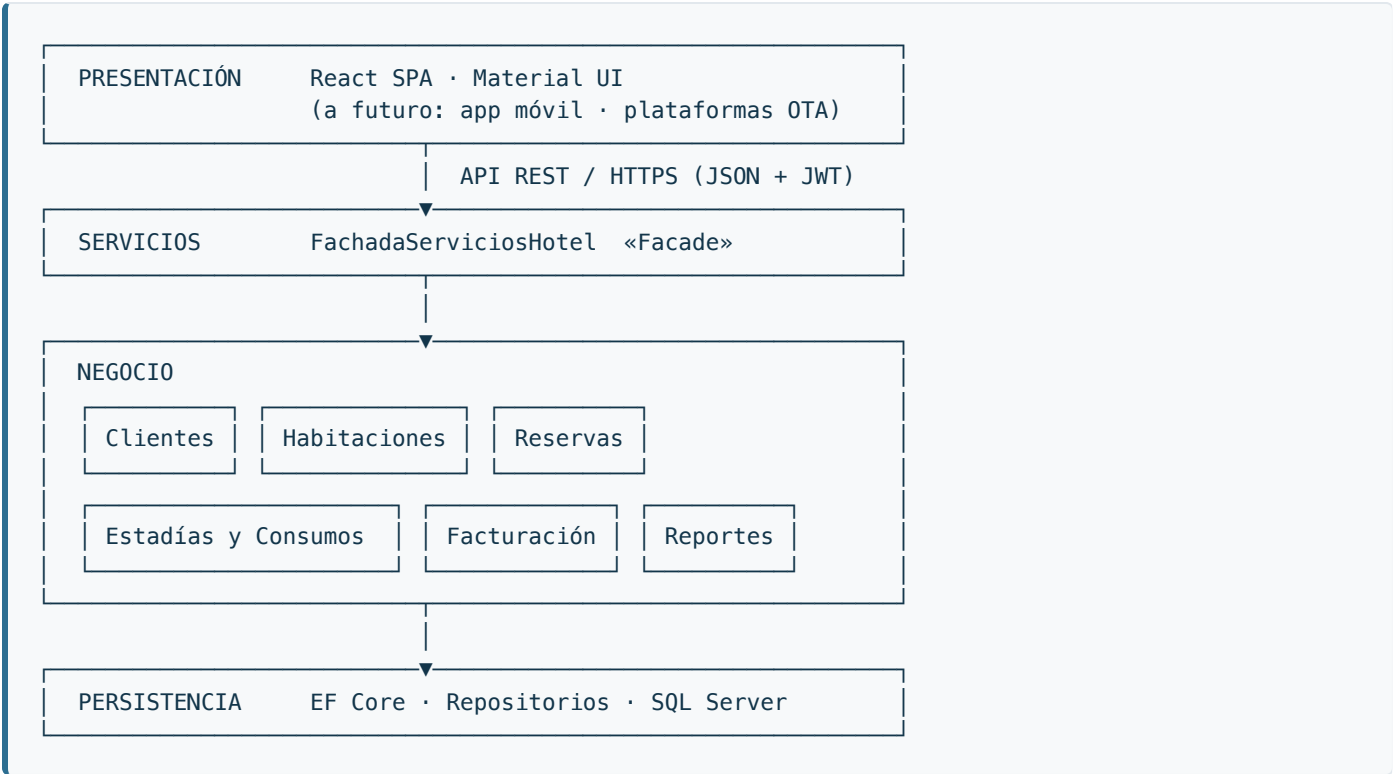
2. Arquitectura

2.1 Arquitectura seleccionada

La Etapa 1 evaluó tres alternativas y seleccionó una **arquitectura en capas con componentes de negocio**:

Alternativa	Resolución
Monolítica	Descartada. Desarrollo inicial rápido, pero el alto acoplamiento dificultaría los cambios a mediano plazo
Microservicios	Descartada. Excelente escalabilidad independiente, pero la complejidad de red y despliegue es sobreingeniería para la primera versión
Capas y componentes	Seleccionada. Responsabilidades delimitadas, componentes internos modificables sin afectar al resto e integración futura sencilla

2.2 Las cuatro capas



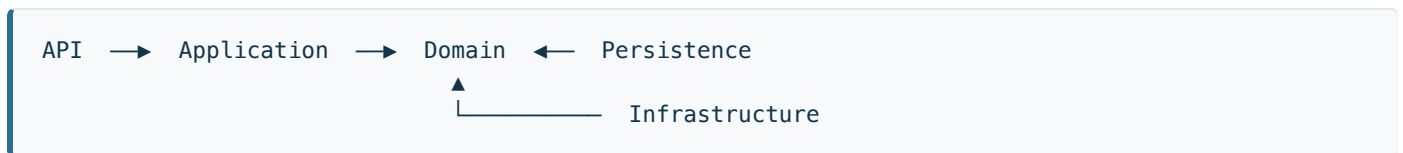
Capa	Responsabilidad
Presentación	Interfaz gráfica y captura de la interacción del usuario
Servicios	Punto único de entrada; canaliza las peticiones hacia la lógica de negocio
Negocio	Reglas del hotel: disponibilidad, tarifas, transiciones de estado, cálculo de facturas
Persistencia	Almacenamiento, mapeo y recuperación de datos

2.3 Materialización sobre Clean Architecture

Las cuatro capas se implementan en siete proyectos:

Proyecto	Capa documental	Contenido
API	Servicios	Controladores REST, middleware, Swagger, autenticación
Application	Servicios y Negocio	Fachada, servicios de aplicación, DTOs, validaciones, mapeadores
Domain	Negocio	Entidades, reglas de negocio, patrones GoF, interfaces de repositorio
Persistence	Persistencia	DbContext , configuraciones, repositorios, migraciones
Infrastructure	Transversal	JWT, hashing, proveedor de fecha, usuario actual
Shared	Transversal	Resultado , paginación, excepciones, constantes
Tests	Transversal	Pruebas unitarias y de integración

Regla de dependencias (vinculante):



Domain no depende de ninguna otra capa. La inversión de dependencias se aplica declarando las interfaces de repositorio en el dominio e implementándolas en persistencia: el negocio nunca conoce Entity Framework ni el motor de base de datos.

2.4 Componentes de negocio

Cada componente declara qué operaciones provee y cuáles requiere de otros:

#	Componente	Provee	Requiere	Patrones
1	Gestión de Clientes	registrar, actualizar, consultar historial, buscar por identificación	—	—
2	Habitaciones y Disponibilidad	consultar disponibilidad, actualizar estado, calcular tarifa	—	Strategy, State
3	Gestión de Reservas	crear, modificar, cancelar, consultar estado	(1) y (2)	State
4	Estadías y Consumos	check-in, check-out, registrar consumo, consultar acumulados	(2) y (3)	Decorator
5	Facturación	generar factura, aplicar descuento, registrar pago	(2) y (4)	Builder
6	Reportes Administrativos	ocupación, ingresos, temporadas	(2), (3) y (5)	Template Method

La capa de Presentación **nunca** invoca a un componente directamente: consume únicamente `IFachadaServiciosHotel`.

2.5 Principios aplicados

SOLID

- **SRP** — `Factura` consolida los cobros y sus cálculos; no se persiste ni se renderiza a sí misma.
- **OCP** — El cálculo de tarifas depende de `IEstrategiaTarifa`: agregar una tarifa nueva no requiere modificar Facturación.
- **LSP** — Cualquier decorador de consumo se procesa de forma transparente en la cadena de la cuenta.
- **ISP** — Las interfaces se segmentan por rol: Recepción no accede a la reportería gerencial.
- **DIP** — La capa de negocio depende de abstracciones, no de implementaciones de persistencia.

Otros: DRY (mapeadores y utilidades centralizados), KISS (sin capas de indirección innecesarias), alta cohesión y bajo acoplamiento entre componentes.

3. Tecnologías y justificación

3.1 Backend

Tecnología	Por qué
ASP.NET Core 8	Versión con soporte a largo plazo, rendimiento alto y ecosistema empresarial maduro
C# 12	Tipado fuerte que favorece un diseño orientado a objetos limpio y detecta errores en compilación
Entity Framework Core 8	Migraciones versionadas, Fluent API y concurrencia optimista nativa, necesaria para RNF03
SQL Server	Motor relacional transaccional exigido por la documentación; soporta el aislamiento serializable que impide sobre-reservas
JWT	Autenticación sin estado: el servidor no guarda sesiones, lo que facilita escalar horizontalmente
Swagger / OpenAPI	Documentación viva y contrato explícito, probable desde el navegador
FluentValidation	Validación declarativa, desacoplada de los modelos y con mensajes en español
AutoMapper 15.1.1	Mapeo entre capas. Se fijó esta versión por seguridad (ver §8.4)
Serilog	Logging estructurado, consultable por propiedades y no solo por texto
Asp.Versioning	Versionado de la API por segmento de URL, para evolucionar sin romper clientes

3.2 Frontend

Tecnología	Por qué
React 19	Interfaz basada en componentes reutilizables
TypeScript	Tipado extremo a extremo: los DTO del backend se replican como tipos y el compilador detecta divergencias
Vite	Arranque casi instantáneo y recarga en caliente
Material UI 7	Sistema de diseño consistente y accesible. Se descartó la versión 9 por cambios incompatibles en su API (ver §6.5)
Axios	Cliente HTTP con interceptores para el token y el manejo global de errores
React Router 7	Enrutamiento declarativo con rutas protegidas por rol
React Hook Form	Formularios de alto rendimiento con validación en tiempo real
TanStack Query	Caché, sincronización e invalidación del estado del servidor
MUI X Charts	Gráficos coherentes con Material UI, sin introducir otra librería visual

4. Base de datos

4.1 Modelo relacional

Once tablas de negocio más el historial de migraciones de Entity Framework.

Tabla	Descripción	PK	FK principales
Usuario	Personal con acceso al sistema	Id	—
Cliente	Huéspedes registrados	Id	—
HistorialCliente	Bitácora de cambios sobre la ficha	Id	ClienteId , UsuarioId
TipoHabitacion	Categorías comerciales y tarifa base	Id	—
Habitacion	Inventario de unidades	Id	TipoHabitacionId
Reserva	Compromiso de hospedaje	Id	ClienteId , HabitacionId , UsuarioRegistroId
Estadia	Permanencia efectiva del huésped	Id	ReservaId , HabitacionId , UsuarioCheckInId , UsuarioCheckOutId
ServicioAdicional	Catálogo de servicios complementarios	Id	—
Consumo	Cargo imputado a una estadía	Id	EstadiaId , ServicioAdicionalId , UsuarioRegistroId
Factura	Comprobante consolidado	Id	EstadiaId , ClienteId , UsuarioEmisionId
DetalleFactura	Línea del comprobante	Id	FacturaId

Cardinalidades relevantes: Cliente 1 → 0..* Reserva · Reserva 1 → 0..1 Estadia · Estadia 1 → 0..* Consumo · Estadia 1 → 0..1 Factura · Factura 1 → 1..* DetalleFactura .

4.2 Índices

34 índices en total. Los más relevantes para el rendimiento (RNF02):

Índice	Propósito
IX_Reserva_Habitacion_Fechas_Estado	Núcleo de la detección de solapamientos; evita recorrer la tabla al verificar disponibilidad
IX_Habitacion_Estado_Tipo	Consulta de disponibilidad filtrada por tipo
IX_Estadia_Habitacion_Estado	Localiza la estadía en curso de una habitación
IX_Factura_FechaEmision_EstadoPago	Reporte de ingresos por rango de fechas
IX_Consumo_Estadia	Consolidación de la cuenta
IX_Cliente_Identificacion (único)	Búsqueda del huésped por documento

4.3 Restricciones

12 restricciones CHECK que garantizan invariantes en el motor y no solo en la aplicación:

Restricción	Regla
CK_Reserva_Fechas	FechaSalida >= FechaEntrada
CK_Reserva_Huespedes	Al menos un huésped
CK_Estadia_FechaCheckOut	El check-out no precede al check-in
CK_Factura_Montos	Ningún importe negativo
CK_Factura_Descuento	El descuento no supera el monto facturado
CK_Consumo_Cantidad	Cantidad mayor que cero
CK_TipoHabitacion_Tarifa	Tarifa no negativa

Además, **17 claves foráneas** con `DELETE RESTRICT` en las relaciones que no admiten cascada, para impedir la pérdida de historial.

4.4 Concurrencia (RNF03)

Las **once tablas** incluyen una columna `rowversion` (`VersionFila`) configurada como token de concurrencia. Si dos recepcionistas modifican el mismo registro, el segundo recibe un conflicto en lugar de sobrescribir en silencio; la API lo traduce a un **HTTP 409** con un mensaje accionable.

La verificación de disponibilidad y la escritura de la reserva ocurren dentro de una misma transacción con nivel de aislamiento **SERIALIZABLE**, que es lo que impide materialmente la sobre-reserva.

4.5 Collation

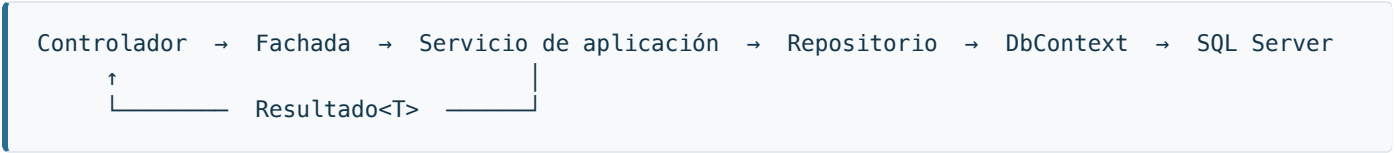
HotelDB y las quince columnas de texto sobre las que el sistema busca usan `Latin1_General_CI_AI`, insensible a mayúsculas y a acentos. Sin ella, buscar «Perez» no encontraría a «Pérez» ni «munoz» a «Muñoz», algo inaceptable en un hotel con huéspedes hispanohablantes. Todo campo de texto emplea `nvarchar`.

4.6 Scripts

Archivo	Contenido
database/01_crear_base_datos.sql	Crea HotelDB con la collation correcta y habilita <code>READ_COMMITTED_SNAPSHOT</code>
database/02_esquema.sql	Script idempotente con tablas, índices, restricciones y claves foráneas

5. Backend

5.1 Flujo de una petición



El controlador no contiene lógica de negocio: recibe el DTO, delega en la fachada y traduce el `Resultado` a HTTP.

5.2 Manejo de errores

Los fallos **previsibles** de negocio no se comunican con excepciones sino con el tipo `Resultado<T>`, que transporta un `TipoError`. `ControladorBase` lo traduce:

TipoError	HTTP	Cuándo
Validacion	400	Datos de entrada inválidos
NoAutenticado	401	Credenciales ausentes o incorrectas
NoAutorizado	403	Rol sin permisos
NoEncontrado	404	El recurso no existe
Conflicto	409	Sobre-reserva, doble facturación, concurrencia
ReglaNegocio	422	Transición de estado inválida u operación fuera de orden

Las excepciones verdaderamente excepcionales las captura `MiddlewareManejoErrores`, que devuelve un `ProblemDetails` con mensaje comprensible. **En producción nunca se expone la traza:** queda solo en el log.

5.3 Endpoints

54 endpoints REST distribuidos en ocho controladores:

Controlador	Ruta base	Endpoints
Autenticación	<code>/api/v1/autenticacion</code>	3
Clientes	<code>/api/v1/clientes</code>	7
Habitaciones	<code>/api/v1/habitaciones</code>	6
Tipos de habitación	<code>/api/v1/tipos-habitacion</code>	3
Reservas	<code>/api/v1/reservas</code>	6
Estadías	<code>/api/v1/estadias</code>	9
Servicios adicionales	<code>/api/v1/servicios-adicionales</code>	3
Facturación	<code>/api/v1/facturas</code>	6
Reportes y panel	<code>/api/v1/reportes</code> , <code>/api/v1/panel</code>	4

Todos exigen autenticación salvo el inicio de sesión y la comprobación de estado `/salud` .

5.4 Repositorios y unidad de trabajo

Cada agregado tiene su repositorio, y `UnidadDeTrabajo` los agrupa sobre un mismo `DbContext` exponiendo `EjecutarEnTransaccionAsync`, que abre la transacción serializable. Las consultas de listado usan `AsNoTracking()` y proyecciones agrupadas para evitar el problema N+1: por ejemplo, el conteo de reservas de todos los clientes de una página se resuelve en una sola consulta.

6. Frontend

6.1 Estructura

```
frontend/src/
├─ api/           Cliente HTTP con interceptores y servicios por componente
├─ componentes/  Piezas reutilizables (KPI, estados, tabla ordenable, confirmación)
├─ contexto/     Sesión del usuario y notificaciones
├─ layout/       Estructura común: barra superior, menú lateral, migas de pan
├─ pantallas/    Las siete pantallas del sistema
├─ tema/         Paleta, tipografía, sombras, duraciones y colores de estado
├─ tipos/        Contratos de la API replicados como tipos de TypeScript
└─ utilidades/   Formato de fechas y moneda, búsqueda diferida
```

6.2 Consumo de la API

`clienteHttp.ts` concentra tres responsabilidades que, repartidas, acabarían duplicadas:

- 1. Adjuntar el token a cada petición.
- 2. Traducir cualquier fallo a un mensaje en español comprensible (`describirError`).
- 3. Cerrar la sesión automáticamente cuando la API responde 401.

Ninguna pantalla construye URLs por su cuenta ni utiliza datos simulados.

6.3 Estado del servidor

TanStack Query gestiona la caché. Tras una mutación se invalidan las claves afectadas, de modo que el panel, el listado de reservas y el estado de las habitaciones se refrescan solos sin que cada pantalla tenga que orquestarlo.

6.4 Experiencia de usuario

Principio (Etapa 2)	Implementación
Consistencia	Menú lateral y barra superior idénticos en todas las pantallas
Visibilidad del estado	Panel con el grid de habitaciones coloreado y leyenda permanente
Prevención de errores	Verificación de disponibilidad con aviso visual antes de confirmar; diálogos de confirmación en operaciones irreversibles
Retroalimentación inmediata	Snackbars tras cada operación; nunca alertas del navegador
Minimizar carga de memoria	Tarjeta del huésped fija durante todo el check-in/check-out
Jerarquía visual	Tarjetas KPI separadas de las tablas; acciones principales destacadas

Estados de carga: esqueletos en las tablas, barra de progreso en recargas, spinners en botones y estados vacíos explicativos con la acción que los resuelve.

Accesibilidad: contraste suficiente, etiquetas en todos los campos, navegación por teclado en elementos interactivos y respeto de `prefers-reduced-motion`.

6.5 Decisiones técnicas

Material UI 7 en lugar de 9. La versión 9 introduce cambios incompatibles: `Stack` deja de aceptar `direction`, `spacing` y `alignItems` como propiedades, y cambian los `styleOverrides` de varios componentes. Con la 9 el proyecto acumulaba 104 errores de tipos ajenos al código propio. Se fijó la 7.3.11, estable y ampliamente documentada.

`strict: true` en TypeScript. La plantilla de Vite no lo incluía y Material UI lo necesita para inferir correctamente sus componentes genéricos.

7. Patrones de diseño

Los seis patrones del catálogo GoF definidos en la Etapa 2, con su implementación real.

7.1 Facade

Ubicación: `Application/Fachada/FachadaServiciosHotel.cs`

Problema. Sin él, la interfaz web, la futura app móvil y las plataformas OTA tendrían que conocer y llamar por separado a los seis componentes de negocio.

Implementación. `IFachadaServiciosHotel` expone los seis componentes tras una superficie única. Añade además dos operaciones compuestas que la Etapa 2 menciona expresamente:

- `CrearReservaAsync(solicitud, confirmar)` — crea la reserva y, si se solicita, la confirma en un solo paso: exactamente lo que hace el botón «Confirmar reserva».
- `GenerarFacturaCheckOutAsync(...)` — cierra la estadía y emite el comprobante, reproduciendo el flujo del diagrama de secuencia de check-out.

Justificación. Responde a RNF05 (mantenibilidad) y RNF07 (integración futura): un cliente nuevo depende de un contrato, no de seis.

7.2 Strategy

Ubicación: `Domain/Patrones/Strategy/`

Problema. El monto de hospedaje varía por tipo de habitación y, a futuro, por temporada o promociones. Codificar esas variantes dentro de Facturación obligaría a modificarla con cada tarifa nueva.

Implementación. `IEstrategiaTarifa` con dos implementaciones: `TarifaEstandar` (noches × tarifa base) y `TarifaTemporadaAlta` (recargo del 25 % en diciembre, enero, febrero y julio). `SelectorEstrategiaTarifa` elige la aplicable al período.

Verificado: una estadía de 3 noches en septiembre a \$85 produce **\$255.00**; el mismo caso en diciembre produce **\$318.75**.

Justificación. Cumple OCP: agregar una tarifa consiste en registrar una implementación más en el contenedor de dependencias.

7.3 State

Ubicación: `Domain/Patrones/State/Habitaciones/` y `Domain/Patrones/State/Reservas/`

Problema. Habitaciones y reservas se comportan de forma distinta según su estado. Resolverlo con condicionales dispersos genera inconsistencias, sobre todo con varios usuarios operando a la vez.

Implementación. Cada estado es una clase que declara a qué otros estados puede transicionar y qué operaciones admite (`PermiteReservar`, `PermiteCheckIn`, `ComprometeHabitacion`). Una transición no contemplada lanza `ExcepcionTransicionEstadoInvalida`, que la API traduce a 422.

- **Habitación:** `Disponible`, `Reservada`, `Ocupada`, `EnLimpieza`, `EnMantenimiento`
- **Reserva:** `Pendiente`, `Confirmada`, `Cancelada`, `Completada`

Justificación. El grafo de transiciones queda explícito y verificable; las 35 pruebas del dominio lo comprueban caso por caso.

7.4 Decorator

Ubicación: `Domain/Patrones/Decorator/`

Problema. Durante la estadía se acumulan consumos de restaurante, lavandería, transporte y actividades. Si `Estadia` tuviera que conocer todas las combinaciones posibles, cada servicio nuevo la obligaría a cambiar.

Implementación. `IComponenteCuenta` lo implementan tanto la cuenta base (`CuentaEstadia`, solo hospedaje) como cada decorador de consumo. `EnsambladorCuenta` envuelve la cuenta con un decorador por cargo; el total y el desglose salen de recorrer la cadena.

Justificación. Añadir un servicio no modifica `Estadia`. La misma cadena alimenta la vista previa de la cuenta y la factura emitida, de modo que ambas cifras no pueden discrepar.

7.5 Builder

Ubicación: `Domain/Patrones/Builder/FacturaBuilder.cs`

Problema. La factura reúne hospedaje, consumos, descuento y método de pago. Sin un punto único de ensamblaje, ese armado quedaría repartido entre varios servicios.

Implementación. `FacturaBuilder` construye por pasos — `ConMontoHospedaje()`, `ConConsumos()`, `ConDescuento()`, `ConPago()` — y `Construir()` valida las precondiciones antes de materializar el comprobante: exige número, detalle de hospedaje y justificación si hay descuento, y rechaza un descuento superior al monto facturado.

Justificación. El orden de las llamadas es indiferente y las invariantes se comprueban en un solo lugar.

7.6 Template Method

Ubicación: `Domain/Patrones/TemplateMethod/`

Problema. Los reportes de ocupación, ingresos y temporadas siguen la misma secuencia —traer datos, filtrar, calcular, dar formato— pero cada uno calcula algo distinto.

Implementación. `ReporteBase.Generar()` fija esa secuencia y es `sealed`: ninguna subclase puede alterarla. Cada reporte implementa solo `ObtenerDatos` y `Calcular`. `ReporteTemporada` redefine además `ConstruirSeries` porque agrupa por mes y no por semana: una temporada no se aprecia en una ventana semanal.

Justificación. El esqueleto común está garantizado; agregar un reporte nuevo consiste en implementar dos métodos.

8. Seguridad

8.1 Autenticación

JWT firmado con **HMAC-SHA256**. El token transporta identificador, nombre de usuario y rol; no contiene datos sensibles porque su contenido es legible por quien lo posea. La clave de firma exige un mínimo de 32 bytes y proviene de variables de entorno o del gestor de secretos: la aplicación **no arranca** si no está configurada.

8.2 Contraseñas

PBKDF2-HMAC-SHA256 con 210 000 iteraciones y sal aleatoria por credencial. El formato almacenado es `iteraciones.sal.hash`, lo que permite elevar el número de iteraciones en el futuro sin invalidar las contraseñas existentes. La verificación usa comparación en tiempo constante para no filtrar información por el tiempo de respuesta.

8.3 Autorización

Dos roles, conforme al diagrama de casos de uso:

Rol	Acceso
Recepcionista	Clientes, habitaciones, reservas, check-in/out, consumos, facturación
Administrador	Todo lo anterior más gestión de usuarios y reportes gerenciales

Aplica el principio ISP: la interfaz de Recepción no alcanza la reportería macro. Existe además una salvaguarda que impide desactivar al último administrador activo o la propia cuenta en sesión.

8.4 Gestión de vulnerabilidades

Durante el desarrollo se detectó que **AutoMapper 12, 13 y 14** arrastraban **CVE-2026-32933** (recursión no controlada, denegación de servicio, CVSS 7.5). Se fijó la versión **15.1.1**, parcheada, lo que obligó a alinear `Microsoft.Extensions.Logging.Abstractions` y `Microsoft.Extensions.Options` a la 10.0.0. La solución compila **sin advertencias de seguridad**.

8.5 Otras medidas

- Ningún secreto en el repositorio: `appsettings.json` contiene solo plantillas vacías.
- CORS restringido a los orígenes declarados en configuración.
- Los logs registran operaciones críticas pero **nunca** contraseñas ni datos sensibles.
- Validación en servidor de toda entrada, con independencia de la validación del frontend.

9. Pruebas

9.1 Cobertura

211 pruebas, 87,0 % de cobertura sobre el código propio (meta de la Etapa 2: 80 %).

Capa	Cobertura
Domain	95,3 %
API	94,2 %
Shared	87,8 %
Persistence	86,2 %
Infrastructure	85,4 %
Application	84,0 %

9.2 Niveles

Carpeta	Cantidad	Nivel	Qué cubre
Dominio/	35	Unitaria	Los seis patrones GoF: transiciones, tarifas, cadena de decoradores, builder, plantilla de reportes
Aplicacion/	16	Unitaria con Moq	Reglas de reservas y de usuarios
Seguridad/	8	Unitaria	PBKDF2, sal aleatoria, hashes mal formados
Api/	10	Unitaria	Middleware de errores: cada excepción a su código HTTP
Integracion/	142	Integración	API ↔ base de datos real, de punta a punta

9.3 Base de datos de pruebas

Las pruebas de integración se ejecutan contra una base **real y dedicada** (`HotelDB_Pruebas`), que la propia suite crea y elimina. No se emplea una base en memoria: ocultaría justamente lo que estas pruebas deben verificar —índices únicos, restricciones CHECK, claves foráneas, concurrencia optimista y collation—.

```
PRUEBAS_SQL_PASSWORD=<contraseña-del-motor> dotnet test backend/HotelParadiseResort.sln
```

9.4 Casos negativos cubiertos

Sobre-reserva → 409 · Check-in sobre reserva pendiente → 422 · Check-in anticipado → 422 · Doble facturación → 409 · Descuento sin justificación → 400 · Recepcionista accediendo a reportes → 403 · Token inválido → 401 · Capacidad de habitación excedida → 422.

10. Instalación

10.1 Requisitos previos

Herramienta	Versión	Verificación
.NET SDK	8.0	<code>dotnet --version</code>
Node.js	20 LTS o superior	<code>node --version</code>
SQL Server	2019 / 2022	—
EF Core Tools	8.0	<code>dotnet ef --version</code>

```
dotnet tool install --global dotnet-ef --version 8.0.11
```

10.2 Base de datos

En Windows, Linux o servidor:

```
docker run --platform linux/amd64 -d --name paradise-sqlserver -e "ACCEPT_EULA=Y" -e "MSSQL_SA_PASSW
```

En macOS con Apple Silicon se emplea Azure SQL Edge por disponer de imagen ARM64 nativa. Ambos usan el mismo proveedor de Entity Framework, por lo que las migraciones y los scripts son idénticos.

10.3 Configuración de secretos

Nunca se colocan credenciales en archivos versionados:

```
dotnet user-secrets set "ConnectionStrings:HotelDB" "Server=localhost,1433;Database=HotelDB;User Id:
```

```
dotnet user-secrets set "Jwt:Clave" "$(openssl rand -base64 48)" --project backend/src/API
```

```
dotnet user-secrets set "Seed:AdminPassword" "TuContraseñaSegura" --project backend/src/API
```

10.4 Migraciones

```
dotnet ef database update --project backend/src/Persistence --startup-project backend/src/API
```

El primer arranque aplica las migraciones pendientes y siembra la cuenta de administrador, los tipos de habitación y el catálogo de servicios.

10.5 Ejecución

```
dotnet run --project backend/src/API
```

```
npm --prefix frontend install && npm --prefix frontend run dev
```

Servicio	Dirección
API	http://localhost:5170
Swagger	http://localhost:5170/swagger
Estado	http://localhost:5170/salud
Interfaz web	http://localhost:5173

11. Despliegue

11.1 Topología

Conforme al diagrama de despliegue de la Etapa 2:

Nodo	Contenido	Comunicación
Estación de trabajo	Navegador con la SPA	HTTPS
Servidor Web	Nginx + build estático	HTTPS/REST (JSON)
Servidor de Aplicaciones	API REST: fachada y componentes	TCP/IP
Servidor de Base de Datos	Motor relacional	—
Ambiente de Staging	Réplica para validar versiones candidatas	Promoción a producción

11.2 Publicación

```
dotnet publish backend/src/API --configuration Release --output ./publish
```

```
npm --prefix frontend run build
```

El frontend genera archivos estáticos en `frontend/dist/`, que se sirven desde el servidor web.

11.3 Variables de entorno en producción

Variable	Descripción
<code>ConnectionStrings__HotelDB</code>	Cadena de conexión
<code>Jwt__Clave</code>	Clave de firma (mínimo 32 caracteres)
<code>Jwt__Emisor</code> , <code>Jwt__Audiencia</code>	Emisor y audiencia del token
<code>Seed__AdminPassword</code>	Contraseña inicial del administrador
<code>ASPNETCORE_ENVIRONMENT</code>	<code>Production</code>
<code>VITE_API_BASE_URL</code>	URL de la API para el frontend

11.4 Integración continua

Cada Pull Request hacia `develop` dispara un flujo que compila, ejecuta las pruebas y verifica la calidad con análisis estático. Ningún cambio llega directo a producción: se despliega primero en Staging.

12. Mantenimiento

12.1 Agregar una tarifa nueva

1. Implementar `IEstrategiaTarifa` en `Domain/Patrones/Strategy/` .
2. Registrarla en `RegistroAplicacion.AgregarAplicacion()` .

No se modifica Facturación ni Reservas: es el patrón Strategy cumpliendo su función.

12.2 Agregar un servicio adicional

Basta darlo de alta desde la pantalla de administración. El `EnsambladorCuenta` ya contempla los cuatro tipos del enunciado; si se incorporara una categoría nueva, se añade el decorador correspondiente y su caso en el ensamblador.

12.3 Agregar un reporte

1. Heredar de `ReporteBase` implementando `ObtenerDatos` y `Calcular` .
2. Exponerlo en `ServicioReportes` y en `ControladorReportes` .

El esqueleto de generación lo aporta el método plantilla.

12.4 Modificar el esquema

Siempre mediante migraciones; nunca editando tablas a mano:

```
dotnet ef migrations add NombreDescriptivo --project backend/src/Persistence --startup-project backe
```

12.5 Convenciones

Ámbito	Convención
C#	<code>PascalCase</code> para tipos y miembros públicos, <code>_camelCase</code> para campos privados, interfaces con <code>I</code> , sufijo <code>Async</code>
React	<code>PascalCase.tsx</code> para componentes, <code>useCamelCase</code> para hooks
Base de datos	Tablas en <code>PascalCase</code> singular, FK <code><Entidad>Id</code> , índices <code>IX_<Tabla>_<Columnas></code>
Git	GitFlow (<code>main</code> , <code>develop</code> , <code>feature/*</code> , <code>hotfix/*</code>) con commits convencionales

12.6 Reglas de código

Nunca: lógica de negocio en controladores, acceso directo a la base desde controladores, código duplicado, consultas N+1, secretos en el código, contraseñas en texto plano ni errores técnicos mostrados al usuario.

Siempre: DTOs, validación de entradas, manejo de excepciones, logging de operaciones críticas sin datos sensibles, paginación, índices y transacciones donde haya concurrencia.

Manual Técnico · Paradise Resort v1.0 · Universidad Latina de Costa Rica